AI Application Development Bootcamp

Day 4: Chatbot with Memory + Project1

AI Memory: Short & Long Term

How LLMs Remember — and How to Build Chatbots That Do Too

A chatbot has no app memory unless we give it one

Memory is not a single feature. It is a design decision: what to store, where to store it, how to retrieve it, and how much to send back.

- LLM context = information inside the current request.
- Application memory = information saved outside the model and reused later.
- Good memory is selective; bad memory blindly sends everything.

Python | Gradio | groq | LangChain | SQLite | ChromaDB

Why Does Memory Matter?

❌ Without Memory

User: "My name is XYZ"

Bot: "Nice to meet you!"

User: "What's my name?"

Bot: "I don't know your name. Each message is independent."

☑ With Memory

User: "My name is XYZ"

Bot: "Nice to meet you, XYZ!"

User: "What's my name?"

Bot: "Your name is XYZ, you told me earlier!"

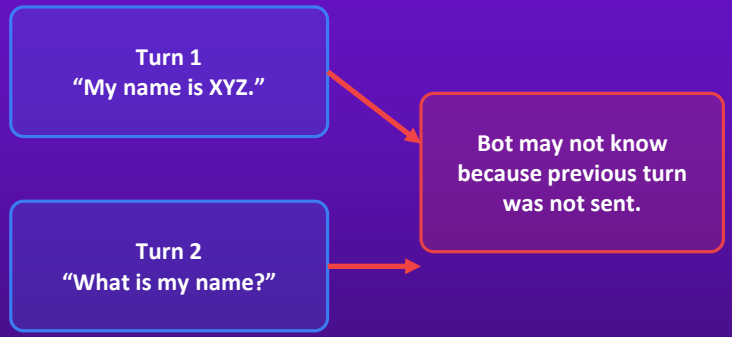
Memory = the ability to maintain context across conversation turns. Without it, every message is treated as a new conversation.

stateless_chatbot.py

stateless chatbot

The baseline : pure input → output with zero memory

```
import os
from groq import Groq



client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

def chat_bot(usr_input):
    chat_completion = client.chat.completions.create(
        messages=[{
            "role": "user",
            "content": usr_input,
            }],
        model="llama-3.1-8b-instant")
    return chat_completion.choices[0].message.content

while True:
    user = input("You: ")
    reply = chat_bot(user)
    print("Bot:", reply)
```



Demo prompt: ask the bot to remember a name, then ask for it in the next request. The failure is the learning moment.

stateless_chatbot_webUI.py

stateless_chatbot_webUI_a.py

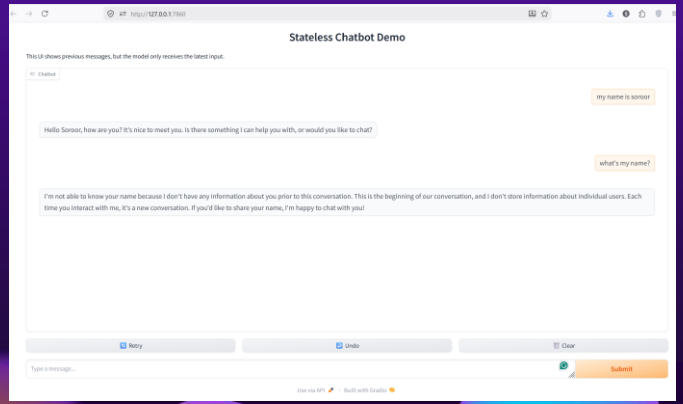
stateless chatbot

Using Gradio to build a simple interface

```
import os
from groq import Groq
import gradio as gr

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

def chat_bot(usr_input):
    chat_completion = client.chat.completions.create(
        messages=[{
            "role": "user",
            "content": usr_input,
            }],
        model="llama-3.1-8b-instant")
    return chat_completion.choices[0].message.content


gr.Interface(fn=chat_bot,
             Inputs="text",
             Outputs="text").launch()
```

gradio

Gradio is an open-source Python library for creating simple and interactive interfaces for machine learning models. It allows you to quickly build web-based user interfaces that enable users to interact with your machine learning models, providing input and receiving model predictions or responses.



Including memory:

Adding conversation history inside the prompt

The simplest memory pattern — append every turn to a list

Short-term memory is usually a buffer: the app sends a list of recent messages with each request.

- Pros: easy, transparent, good for immediate continuity.
- Cons: token cost increases, stale context may distract the model.

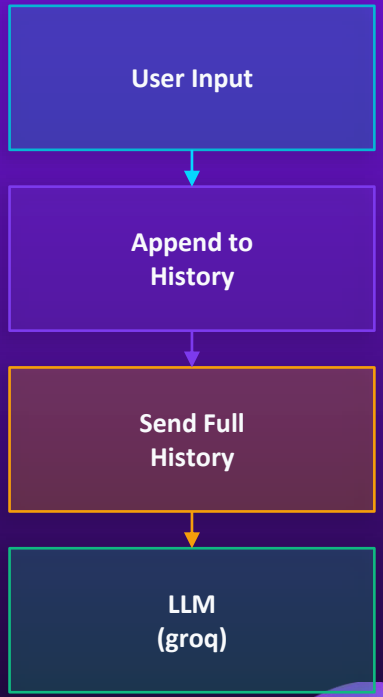
- Common strategy: keep the last N turns + important facts.

basicM_chatbot_webUI_a.py

Add basic memory (pass the history back in)

Now the bot can answer follow-up questions from recent turns.

```
memory = []

def chat_bot(usr_input, history):
    memory.append({
        "role": "user",
        "content": usr_input
    })

    chat_completion = client.chat.completions.create(
        messages=memory,
        model="llama-3.1-8b-instant"
    )
    reply = chat_completion.choices[0].message.content

    # save bot reply
    memory.append({
        "role": "assistant",
        "content": reply
    })

    return reply
```



Adding conversation history

[the growing context problem]

Simply appending every turn to the prompt grows very fast and can affect the model's response.

Common Solutions:

Pattern	What it stores	When to use it
Conversation history	Keep recent turns verbatim	Best for follow-up coherence
Summarisation	Compress old turns into a running summary	Best when conversations become long
Knowledge extraction	Store durable facts/preferences as structured records	Best when specific facts matter later

basicM_chatbot_recentH_webUI_a.py

Recent Conversation History

```
memory = []

def chat_bot(prompt, history):
    memory.append({"role": "user", "content": prompt})

    # keep only last 6 messages
    recent_memory = memory[-6:]

    chat_completion = client.chat.completions.create(
        messages=recent_memory,
        model="llama-3.1-8b-instant"
    )

    reply = chat_completion.choices[0].message.content

    memory.append({"role": "assistant", "content": reply})

    return reply
```

basicM_chatbot_summary_webUI_a.py

Summarization

```
memory = []
summary = ""

def summarize_memory():
    global memory, summary

    text = "\n".join([f"{m['role']}: {m['content']}" for m
in memory])

    response = client.chat.completions.create(
        messages=[
            {"role": "user", "content": f"Summarize this
conversation briefly:\n{text}"}
        ],
        model="llama-3.1-8b-instant"
    )

    summary = response.choices[0].message.content
    memory = []
```

Long-term memory
(stored outside the context window)

Saved interactions can be reused later, but must be selected carefully.

- Raw history: every user/assistant message.
- Summaries: compressed versions of old conversations.
- Facts/preferences: extracted durable details.
- Documents/skills: project files like skills.md.

Long-term memory is storage + retrieval. Storage alone is not enough.

Good design question: “What should be included this turn, and what should stay outside?”

Storage Backends

Where should we store memory?

JSON File

```
import json
json.dump(history,
  open("chat.json","w"))
```

✓ PROS

- Zero dependencies
- Human-readable
- Easy to debug

✗ CONS

- Slow on large files
- No querying
- No concurrent writes

docs.python.org/json

Markdown

```
# Day 4 Chat
## Turn 1
**User:** Hi
**Bot:** Hello!
```

✓ PROS

- Human-readable
- Version-controllable
- skills.md compatible

✗ CONS

- Need custom parser
- No structure
- Poor retrieval

commonmark.org

SQLite

```
CREATE TABLE history
  (id, role, content,
   timestamp);
```

✓ PROS

- SQL queries!
- Fast retrieval
- Concurrent safe

✗ CONS

- More setup
- Binary format
- Overkill for tiny bots

sqlite.org/docs.html

Json_chatbot_webUI_a.py

Json

```
FILE = "chat.json"

def load_memory():
    ***

def save_memory(memory):
    ***

def chat_bot(prompt, history):
    memory = load_memory()

    memory.append({
        "role": "user",
        "content": prompt
    })

    response = client.chat.completions.create(
        messages=memory,
        model="llama-3.1-8b-instant"
    )

    reply = response.choices[0].message.content

    memory.append({
        "role": "assistant",
        "content": reply
    })

    save_memory(memory)

    return reply
```

MD_chatbot_webUI_a.py

Mark Down

```
FILE = "memory.md"

def load_memory():
    ***

def save_memory(prompt, reply):
    ***

def chat_bot(prompt, history):
    memory_text = load_memory()

    response = client.chat.completions.create(
        messages=[
            {
                "role": "system",
                "content": f"Previous memory:\n{memory_text}"
            },
            {
                "role": "user",
                "content": prompt
            }
        ],
        model="llama-3.1-8b-instant"
    )

    reply = response.choices[0].message.content

    save_memory(prompt, reply)

    return reply
```

Json+MD_chatbot_webUI_a.py

JSON+ Markdown Hybrid

Simple Conversation Memory

Loads entire file before searching → not ideal at scale

```
profile = load_md()
chat_history = load_json()

messages = [
    {"role": "system", "content": profile}
]

messages.extend(chat_history)
```

Structured Format

Stores chat as clean key-value pairs

Easy Debugging

Open file and inspect conversations easily

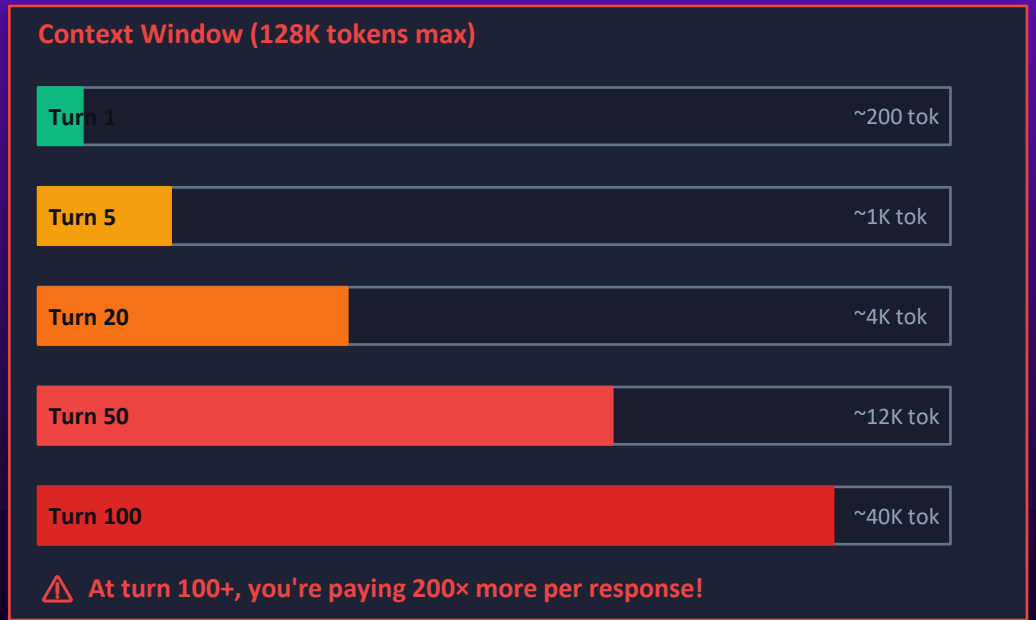
Good for Prototypes

Perfect for small chatbot demos

[the growing context problem]

As memory grows, naive history stuffing becomes slower and noisier.

- More tokens in → higher cost and latency.
- More old context → more chance of irrelevant distractions.
- More retrieved snippets → risk of conflicting evidence.
- Better strategy → select, compress, and retrieve.



What Goes Wrong

Cost explodes

Tokens = $$$. 10K tokens per call gets expensive fast

Latency rises

More tokens = slower inference. UX degrades

Lost in the middle

LLMs struggle with very long contexts — early info fades

Noisy responses

Irrelevant old messages pollute the context

Hard limit hit

GPT-4o: 128K tokens. Exceed it → API error

Memory Management Strategies

Conversation History

Beginner

HOW

Append every message to a list and send it all

BEST FOR

Short sessions, prototypes, demos

LIMITATION

Token limit hit after ~50-100 turns

Summarisation

Intermediate

HOW

Compress old history when approaching token limit

BEST FOR

Longer sessions, moderate detail needed

LIMITATION

Lossy — detail is permanently gone

Knowledge Extraction

Advanced

HOW

Extract key facts to a structured store (DB/Vector)

BEST FOR

Long-running assistants, multi-session memory

LIMITATION

Complex to implement correctly

A simple SQLite schema for chat memory

```
CREATE TABLE messages (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  session_id TEXT NOT NULL,
  role TEXT CHECK(role IN ('user','assistant')),
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_messages_session
ON messages(session_id, created_at);
```

- session_id separates conversations.
- role lets the prompt builder reconstruct message order.
- created_at supports recency filtering.
- content can later be indexed using FTS5 or embedded for semantic search.

Design tip:
store raw history first. Add summaries/facts in separate tables when the need becomes clear.

Knowledge extraction

Instead of remembering every sentence, extract useful facts into structured memory.

```
Example extracted memory record
{
  "type": "preference",
  "key": "ui_style",
  "value": "student prefers Gradio UI examples",
  "source_message_id": 42,
  "confidence": 0.86,
  "updated_at": "2026-04-28"
}
```

- Good for user preferences, project decisions, names, constraints, and selected learning progress.
- Danger: do not extract everything; stale or wrong facts can harm the chatbot.
- Always consider privacy, consent, and the ability to correct/delete memory.

Note, Keyword search only matches words, not meaning.

SQLite_chatbot_webUI_a.py

SQLite: Searchable Chat Memory

Stores messages as rows; Query only what you need

```
import sqlite3

conn = sqlite3.connect("memory.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS history (
    id INTEGER PRIMARY KEY,
    role TEXT,
    content TEXT
)
""")

def save_memory(role, content):
    cursor.execute(
        "INSERT INTO history (role, content) VALUES (?, ?)",
        (role, content)
    )
    conn.commit()

def search_memory(keyword):
    cursor.execute(
        "SELECT content FROM history WHERE content LIKE ?",
        (f"%{keyword}%",)
    )
    rows = cursor.fetchall()
    return "\n".join([r[0] for r in rows])
```

Structured Storage

Each message becomes one database row.
Support multiple users / conversations simultaneously.

Keyword Search

Find matching memory using LIKE
& Retrieve only matching rows instead of loading all history.

Persistent Storage

Survives server restarts — memory lives forever

Scalable

Handles millions of messages, still blazing fast

Keyword Search (and Its Limits)

How Keyword Search Works

```
def search_history(keyword, conn):
    cur = conn.execute("""
        SELECT role, content FROM history
        WHERE content LIKE ?
        ORDER BY ts DESC LIMIT 5""",
        [f"%{keyword}%"])
    return cur.fetchall()
```

Where Keyword Search Fails

- User says "car" → won't find "automobile"
- User says "sick" → won't find "unwell"
- Typos: "Pyhton" → won't find "Python"
- No ranking — all results are equal
- Context-blind — single word vs full meaning

Comparison: Keyword vs Semantic Search

Criteria	Keyword (LIKE)	Semantic (Vector DB)
Setup complexity	Trivial	Moderate
Synonym handling	None	Built-in
Query: 'show me my car msg'	Miss 'automobile'	Finds it
Speed on large dataset	Slow (table scan)	Fast (index)
Dependencies	None	chromadb, sentence-transformers

Vector DBs & Semantic Search (Search by meaning)

Embeddings map text into vectors so similar meanings are near each other.

Why Vectors Work

- "car" and "automobile" are CLOSE in vector space
- "car" and "pizza" are FAR in vector space
- Meaning is encoded — not just spelling
- Cosine similarity finds semantic neighbors
- Multilingual: 'coche' ≈ 'car' in Spanish

- Embed each memory or document chunk.
- Embed the user query.
- Find nearest vectors, not exact words.
- Return the matched text to the prompt.

Keyword search: “same words?”
Semantic search: “same meaning?”

*** Chroma preview only today; full RAG session later.**

Vector DBs & Semantic Search (Search by meaning)

Embeddings map text into vectors so similar meanings are near each other.

```
import chromadb
client = chromadb.PersistentClient(path='./chroma_db')
collection = client.get_or_create_collection('memory')

collection.add(
    documents=[message_text],
    metadatas=[{'session_id': session_id, 'role': role}],
    ids=[message_id]
)

results = collection.query(
    query_texts=[user_question],
    n_results=3
)
```

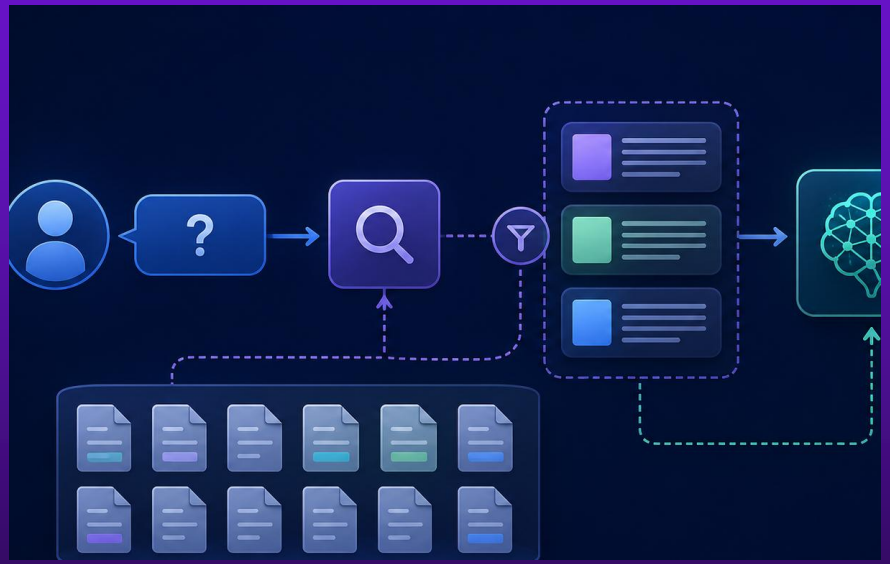
- Embed each memory or document chunk.
- Embed the user query.
- Find nearest vectors, not exact words.
- Return the matched text to the prompt.

Keyword search: “same words?”
Semantic search: “same meaning?”

SQLite_Semantic_chatbot_webUI_a.py

ChromaDB_chatbot_Semantic_webUI_a.py

RAG: retrieve relevant parts instead of sending everything



- Memory problem: “Which previous pieces should I send?”
- RAG answer: retrieve the relevant pieces first.
- Then send only those pieces with the user question.
- The same pattern works for chat history, skills.md, PDFs, and websites.

A memory-aware chatbot is an app design, not just a model call.

Your turn!

Build Your Own Context-Aware Chatbot

- Build a working Gradio or CLI chatbot
- Maintain conversation history
- Store memory using JSON / SQLite / ChromaDB
- Retrieve relevant memory using keyword OR semantic search
- Use the provided skills.md and .json files from Moodle.
- Apply prompt engineering to improve reliability

Bonus:

- Add summarization
- Support multiple users with separate memory
- Improve latency/performance